

Advanced Data Handling Exercises

8 Activities to consolidate concepts

You will practice — covering generators, lazy iterators, itertools streaming, dataclass schema validation, safe custom serializers, chunked file processing, vectorized pandas columns, and memory-profiling diagnostics.

EXERCISE 1

Lazy Data Loader Generator

- **Objective** : Refactor a memory-intensive list-based dataset loader into an efficient generator pipeline.
- Inspect the baseline list-based loader in the template code.
- Define a generator function using the yield keyword to stream elements individually.
- Ensure that processing operations occur lazily on demand during iteration rather than storing items in intermediate lists.
- Verify output correctness and run validation on both positive and negative elements.

ex1_template.py

```
"""
Exercise 1: Generators, Iterators and Lazy Evaluation
Task: Refactor a list loader into a lazy streaming pipeline.
"""

# Baseline list-based data loader (materializes everything)
def load_all_records_baseline(data_list):
    results = []
    for item in data_list:
        # Simulate processing and filtering
        if item > 0:
            results.append(item * 10)
    return results

# TODO: Refactor the code above into a lazy generator function
def stream_records_lazy(data_list):
    """
    Yields processed numbers one at a time, lazily.
    """
    # Implement lazy evaluation using yield here
    pass

# Verification
if __name__ == "__main__":
    raw_data = [5, -2, 12, 0, 8, -1]
    baseline_out = load_all_records_baseline(raw_data)
    print(f"Baseline Output (List): {baseline_out}")
    lazy_gen = stream_records_lazy(raw_data)
    print(f"Is generator: {hasattr(lazy_gen, '__next__')}")
    if lazy_gen is not None:
        try:
            print(f"Lazy Output (Stream): {list(lazy_gen)}")
        except Exception as e:
            print(f"Error executing lazy generator: {e}")
```

EXERCISE 1 · SOLUTION

Lazy Data Loader Generator

- The baseline loader builds a full results list before returning anything — every item sits in memory at once, even for a dataset with a million rows.
- Replacing return with yield turns the function into a generator: each processed value is produced one at a time, on demand.
- Calling the generator function does not run any code yet — it only starts executing when something iterates it (here, `list(lazy_gen)`).
- The filtering condition (`item > 0`) and the transformation (`item * 10`) happen lazily, per element, instead of in one eager pass.

ex1_lazy_pipeline.py

```
def load_all_records_baseline(data_list):
    results = []
    for item in data_list:
        if item > 0:
            results.append(item * 10)
    return results # everything materialised before returning

def stream_records_lazy(data_list):
    # yield makes this a generator: nothing runs until iterated
    for item in data_list:
        if item > 0:
            yield item * 10 # one value produced at a time

raw_data = [5, -2, 12, 0, 8, -1]
print(load_all_records_baseline(raw_data))

lazy_gen = stream_records_lazy(raw_data)
print(hasattr(lazy_gen, "__next__")) # True: it's an iterator
print(list(lazy_gen)) # consuming it runs the generator body
```

EXPECTED SHELL OUTPUT

```
Baseline Output (List): [50, 120, 80]
Is generator: True
Lazy Output (Stream): [50, 120, 80]
```

EXERCISE 2

Streaming with itertools

- **Objective** : Filter, group, and slice dynamic data streams lazily without list copies using itertools.
- Study the baseline grouping function which uses intermediate slicing arrays.
- Refactor the pipeline to use islice to extract elements dynamically inside grouping iterations.
- Ensure that group iterators are handled lazily without full list materializations.
- Verify that calculations match the baseline output exactly.

```

"""
Exercise 2: itertools for Efficient Data Pipelines
Task: Process records with itertools, not intermediate lists.
"""

from itertools import islice, groupby, accumulate

# TODO: Refactor this list-based processor using itertools
def process_data_pipeline_baseline(raw_records):
    sorted_records = sorted(
        raw_records, key=lambda x: x["category"])
    grouped_totals = {}
    for key, group in groupby(
        sorted_records, key=lambda x: x["category"]):
        group_list = list(group)
        slice_items = group_list[:3]
        total_sum = sum(item["val"] for item in slice_items)
        grouped_totals[key] = total_sum
    return grouped_totals

# TODO: Build a lazy transformation stream using islice and groupby
def process_data_pipeline_itertools(raw_records):
    """
    Sorts, groups, slices first 3 elements of each group, and
    returns a mapping of categories to their sliced aggregates.
    """
    # Hint: slice lazily using islice inside the grouping loop!
    pass

if __name__ == "__main__":
    records = [
        {"category": "A", "val": 10},
        {"category": "A", "val": 20},
        {"category": "A", "val": 30},
        {"category": "A", "val": 40},
        {"category": "B", "val": 5},
        {"category": "B", "val": 15},
        {"category": "B", "val": 25},
        {"category": "B", "val": 35},
    ]
    print(f"Baseline: {process_data_pipeline_baseline(records)}")
    itertools_out = process_data_pipeline_itertools(records)
    print(f"Itertools: {itertools_out}")

```

EXERCISE 2 · SOLUTION

Streaming with itertools

- The baseline groups records with `groupby`, but then calls `list(group)` to slice the first three items — materialising every group in full just to take a few elements.
- `itertools.islice(group, 3)` takes the first three items directly from the group iterator, without ever building the full list.
- `groupby` only groups consecutive items, so the data must be sorted by the grouping key first — both versions do this identically.
- Both versions must agree exactly: the `itertools` refactor changes how memory is used, not what the answer is.

ex2_itertools_pipeline.py

```
from itertools import islice, groupby

def process_data_pipeline_itertools(raw_records):
    # groupby only groups *consecutive* items, so sort by key first
    sorted_records = sorted(
        raw_records, key=lambda x: x["category"])
    grouped_totals = {}
    for key, group in groupby(
        sorted_records, key=lambda x: x["category"]):
        # islice pulls the first 3 items directly from the group
        # iterator -- no intermediate list is ever built
        first_three = islice(group, 3)
        grouped_totals[key] = sum(
            item["val"] for item in first_three)
    return grouped_totals

records = [
    {"category": "A", "val": 10},
    {"category": "A", "val": 20},
    {"category": "A", "val": 30},
    {"category": "A", "val": 40},
    {"category": "B", "val": 5},
    {"category": "B", "val": 15},
    {"category": "B", "val": 25},
    {"category": "B", "val": 35},
]
print(process_data_pipeline_itertools(records))
```

EXPECTED SHELL OUTPUT

`{'A': 60, 'B': 45}` (matches the list-slicing baseline exactly)

EXERCISE 3

Validated Dataclasses

- **Objective** : Design an OrderRecord dataclass schema that intercepts and validates values on instantiation.
- Declare an OrderRecord dataclass matching the requested fields.
- Implement validation rules inside the dunder `__post_init__` method.
- Reject records by raising `ValueError` if the price or quantity is zero or negative, or if the customer ID is empty.
- Verify that validation successfully blocks corrupt records during testing.

ex3_template.py

```
"""
Exercise 3: Structured Data with Dataclasses and Validation
Task: Create a validated OrderRecord dataclass.
"""

from dataclasses import import dataclass

# TODO: Implement ordered record fields with post-init validation
@dataclass
class OrderRecord:
    order_id: str
    price: float
    quantity: int
    customer_id: str

    def __post_init__(self):
        """
        Validate fields:
        - Price must be greater than zero.
        - Quantity must be greater than zero.
        - customer_id must not be empty.
        """
        # Add your parameter validation guards here
        pass

if __name__ == "__main__":
    try:
        # 1. Valid Record
        valid = OrderRecord("ORD_001", 15.99, 3, "CUST_99")
        print(f"Order created successfully: {valid}")
        # 2. Invalid Record (should raise ValueError)
        invalid = OrderRecord("ORD_002", -5.00, 2, "CUST_99")
    except ValueError as e:
        print(f"Validation caught error: {e}")
```

EXERCISE 3 · SOLUTION

Validated Dataclasses

- `__post_init__` runs automatically right after a dataclass's generated `__init__` finishes — it is the natural place to validate fields.
- Raising `ValueError` inside `__post_init__` stops the object from being created at all: an invalid `OrderRecord` never exists, rather than existing in a broken state.
- Three independent guards check price, quantity, and `customer_id` — each raises its own clear message naming exactly what failed.
- This turns "a dict with the right keys" into "a value that is guaranteed valid the moment it exists."

ex3_order_validation.py

```
from dataclasses import dataclass

@dataclass
class OrderRecord:
    order_id: str
    price: float
    quantity: int
    customer_id: str

    def __post_init__(self):
        # runs automatically right after the generated __init__
        if self.price <= 0:
            raise ValueError("price must be positive.")
        if self.quantity <= 0:
            raise ValueError("quantity must be positive.")
        if not self.customer_id:
            raise ValueError("customer_id must not be empty.")

valid = OrderRecord("ORD_001", 15.99, 3, "CUST_99")
print(f"Order created successfully: {valid}")

try:
    invalid = OrderRecord("ORD_002", -5.00, 2, "CUST_99")
except ValueError as e:
    print(f"Validation caught error: {e}")
```

EXPECTED SHELL OUTPUT

```
Order created successfully: OrderRecord(...)
Validation caught error: price must be positive.
```

EXERCISE 4

Custom JSON Encoding

- **Objective** : Build a custom JSONEncoder layer capable of serializing structured dataclass instances.
- Implement a custom class subclassing json.JSONEncoder.
- Override default() to recognize and unpack dataclass structures into serializable dictionaries using asdict.
- Verify that serialized strings can be loaded back safely and restored as OrderRecord instances.
- Test exception scenarios to ensure default serialization behavior is preserved for base types.

ex4_template.py

```
"""
Exercise 4: Safe Serialization with Custom JSON Encoders
Task: Build a custom JSONEncoder for OrderRecord instances.
"""

import json
from dataclasses import dataclass, asdict

@dataclass
class OrderRecord:
    order_id: str
    price: float
    quantity: int

# TODO: Implement a custom JSON Encoder class
class OrderEncoder(json.JSONEncoder):
    def default(self, obj):
        # Hint: if obj is a dataclass instance, use asdict()
        pass

if __name__ == "__main__":
    order = OrderRecord("ORD_101", 45.50, 10)
    try:
        # json.dumps fails on custom classes without an encoder
        payload = json.dumps(order, cls=OrderEncoder)
        print(f"Encoded JSON: {payload}")
        decoded_dict = json.loads(payload)
        print(f"Decoded dictionary: {decoded_dict}")
    except TypeError as e:
        print(f"Type error during serialization: {e}")
```


EXERCISE 4 · SOLUTION

Custom JSON Encoding

- `json.dumps()` has no idea how to serialise a dataclass instance by default — it only understands plain dicts, lists, strings, and numbers.
- Subclassing `json.JSONEncoder` and overriding `default()` lets you intercept exactly the objects the built-in encoder doesn't recognise.
- `dataclasses.asdict()` converts a dataclass instance into a plain dict, which the encoder can then serialise normally.
- Calling `super().default(obj)` for anything that isn't a dataclass preserves the built-in encoder's normal error behaviour for genuinely unsupported types.

ex4_safe_json_encoder.py

```
import json
from dataclasses import dataclass, asdict, is_dataclass

@dataclass
class OrderRecord:
    order_id: str
    price: float
    quantity: int

class OrderEncoder(json.JSONEncoder):
    def default(self, obj):
        if is_dataclass(obj):
            # convert to a plain, serialisable dict
            return asdict(obj)
        # preserve normal error behaviour for other types
        return super().default(obj)

order = OrderRecord("ORD_101", 45.50, 10)
payload = json.dumps(order, cls=OrderEncoder)
print(f"Encoded JSON: {payload}")

decoded_dict = json.loads(payload)
restored = OrderRecord(**decoded_dict) # rebuild the dataclass
print(f"Restored Record Object: {restored}")
```

EXPECTED SHELL OUTPUT

```
{"order_id": "ORD_101", "price": 45.5, "quantity": 10}
Restored: OrderRecord(...)
```

EXERCISE 5

Chunked File Streaming

- **Objective** : Read and aggregate values from large files line-by-line using streaming iterators.
- Construct a generator function that opens a file and yields records incrementally using `csv.DictReader`.
- Aggregate column summaries row-by-row on the fly to avoid loading full columns into memory.
- Ensure that resource handles are closed safely after processing is finished.
- Verify aggregate outcomes against standard test records.

```
"""
Exercise 5: Chunked Processing of Large Files
Task: Process a large CSV line by line to sum categories.
"""

import csv
import os

# TODO: Implement an incremental CSV reader generator
def stream_csv_records(file_path):
    """Yields rows incrementally from the CSV file as dict records."""
    pass

if __name__ == "__main__":
    csv_file = "raw_data.csv"
    with open(csv_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["category", "amount"])
        writer.writerow(["hardware", "150.00"])
        writer.writerow(["software", "49.99"])
        writer.writerow(["hardware", "350.00"])
        writer.writerow(["software", "150.00"])
    totals = {}
    records_stream = stream_csv_records(csv_file)
    if records_stream is not None:
        for row in records_stream:
            cat = row["category"]
            amt = float(row["amount"])
            totals[cat] = totals.get(cat, 0.0) + amt
        print(f"Final aggregated totals: {totals}")
    os.remove(csv_file)
```

EXERCISE 5 · SOLUTION

Chunked File Streaming

- `csv.DictReader` already reads one row at a time from disk — wrapping it in a generator (`yield row`) preserves that row-by-row behaviour all the way out to the caller.
- Opening the file with a `with` block guarantees the file handle closes automatically, even if the caller stops iterating early.
- The running totals dictionary is updated one row at a time as rows stream in — the full column of amounts is never held in memory together.
- This scales the same way whether the file has 4 rows or 4 million: memory use depends on the totals dict, not the file size.

ex5_csv_stream.py

```
import csv

def stream_csv_records(file_path):
    # "with" guarantees the file closes on early exit too
    with open(file_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            yield row # one row at a time, not the whole file

totals = {}
for row in stream_csv_records("raw_data.csv"):
    cat = row["category"]
    amt = float(row["amount"])
    totals[cat] = totals.get(cat, 0.0) + amt # running total
print(f"Final aggregated totals: {totals}")
```

EXPECTED SHELL OUTPUT

```
Final aggregated totals: {'hardware': 500.0, 'software': 199.99}
```

EXERCISE 6

Vectorized pandas

- **Objective** : Convert slow row loops into high-performance column-wise vectorized expressions.
- Analyze the baseline loop which uses iterrows to modify columns.
- Refactor the code into column math expressions (avoid loops entirely!).
- Apply groupby transform to generate a column of category total sums aligned with rows.
- Calculate each row's percentage contribution using division operations on columns.

ex6_template.py

```
"""
Exercise 6: Advanced pandas for Efficient Manipulation
Task: Optimize a loop-based pandas operation into vector ops.
"""

import pandas as pd

def calculate_metrics_loop(df):
    results = []
    for idx, row in df.iterrows():
        # Slow row-by-row logic
        net_amount = row["amount"] * 0.90
        results.append(net_amount)
    df["net_amount"] = results
    return df

# TODO: Refactor using column vectorization and transform
def calculate_metrics_vectorized(df):
    """
    - net_amount = amount * 0.90
    - category_total = total net_amount per category
    - percentage_share = each row's share of its category total
    """
    # Avoid iterrows! Use column expressions and groupby.transform
    pass

if __name__ == "__main__":
    df_data = pd.DataFrame({
        "category": ["A", "A", "B", "B", "C"],
        "amount": [100.0, 200.0, 150.0, 50.0, 300.0],
    })
    print(calculate_metrics_loop(df_data.copy()))
    vectorized_df = calculate_metrics_vectorized(df_data.copy())
    print(vectorized_df)
```

EXERCISE 6 · SOLUTION

Vectorized pandas

- `df.iterrows()` loops through a DataFrame one Python-level row at a time — it is convenient, but it forfeits pandas' compiled, columnar speed entirely.
- `df["amount"] * 0.90` performs the same multiplication on the whole column at once, in compiled C code rather than a Python loop.
- `groupby("category").transform("sum")` computes each group's total and re-aligns it back onto every row of that group — no manual merge needed.
- Dividing one column by another (`net_amount / category_total`) computes every row's percentage share in a single vectorized expression.

ex6_pandas_vectorized.py

```
import pandas as pd

def calculate_metrics_vectorized(df):
    # column-wise arithmetic runs on the whole column at once
    df["net_amount"] = df["amount"] * 0.90
    # transform("sum") aligns each group's total onto every row
    grp = df.groupby("category")["net_amount"]
    df["category_total"] = grp.transform("sum")
    # column divided by column: vectorized, no loop needed
    df["percentage_share"] = (
        df["net_amount"] / df["category_total"])
    return df

df_data = pd.DataFrame({
    "category": ["A", "A", "B", "B", "C"],
    "amount": [100.0, 200.0, 150.0, 50.0, 300.0],
})
print(calculate_metrics_vectorized(df_data))
```

EXPECTED SHELL OUTPUT

```
category_total: A=270.0, B=180.0, C=270.0
percentage_share sums to 1.0 within each category
```

EXERCISE 7

Memory Efficiency

- **Objective** : Reduce a DataFrame's heap memory usage by casting columns to compact dtypes.
- Review the baseline unoptimized dataframe dtypes (int64, float64, and object strings).
- Downcast status_code to integer 16-bit (int16) and temperature to float 32-bit (float32) representation.
- Convert the repeated string column (region_label) to a categorical datatype.
- Profile memory footprints before and after optimization, calculating total memory savings.

ex7_template.py

```
"""
Exercise 7: Memory Management and dtype Specifications
Task: Optimize a memory-heavy DataFrame with compact dtypes.
"""

import pandas as pd

def create_unoptimized_df():
    data = {
        "status_code": [200, 404, 500, 200] * 1000,
        "region_label": ["north_america", "europe", "asia",
                        "south_america"] * 1000,
        "temperature": [25.5, 12.0, 30.2, 18.1] * 1000,
    }
    return pd.DataFrame(data)

# TODO: Implement a memory-optimized converter
def optimize_dataframe_dtypes(df):
    """
    - region_label -> 'category'
    - status_code -> 'int16'
    - temperature -> 'float32'
    """
    pass

if __name__ == "__main__":
    df_raw = create_unoptimized_df()
    mem_before = df_raw.memory_usage(deep=True).sum()
    print(f"Before: {mem_before / 1024:.2f} KB")
    df_opt = optimize_dataframe_dtypes(df_raw.copy())
    if df_opt is not None:
        mem_after = df_opt.memory_usage(deep=True).sum()
        print(f"After: {mem_after / 1024:.2f} KB")
        savings = (mem_before - mem_after) / mem_before * 100
        print(f"Savings: {savings:.2f}%")
```

EXERCISE 7 · SOLUTION

Memory Efficiency

- pandas defaults are generous: whole numbers become int64, decimals become float64, and repeated strings become the generic, memory-heavy object dtype.
- `astype("int16")` keeps the same whole numbers in a quarter of the space — fine as long as the values fit comfortably within int16's range.
- `astype("category")` is the biggest win for repeated strings: pandas stores each unique string once and reuses a small integer code for every repeat.
- `memory_usage(deep=True)` is essential for measuring object-dtype columns honestly — without `deep=True`, string columns are badly under-reported.

ex7_memory_optimization.py

```
import pandas as pd

def optimize_dataframe_dtypes(df):
    # store each unique string once, reuse a small code
    df["region_label"] = df["region_label"].astype("category")
    # whole numbers that comfortably fit in a smaller range
    df["status_code"] = df["status_code"].astype("int16")
    # decimals that don't need float64's extra precision
    df["temperature"] = df["temperature"].astype("float32")
    return df

df_raw = create_unoptimized_df()
mem_before = df_raw.memory_usage(deep=True).sum()
# deep=True gives honest sizing for object/string columns
df_opt = optimize_dataframe_dtypes(df_raw.copy())
mem_after = df_opt.memory_usage(deep=True).sum()
savings = (mem_before - mem_after) / mem_before * 100
print(f"Before: {mem_before/1024:.2f} KB")
print(f"After: {mem_after/1024:.2f} KB")
print(f"Savings: {savings:.2f}%")
```

EXPECTED SHELL OUTPUT

Before: 289.19 KB → After: 27.70 KB → Savings: 90.42%

EXERCISE 8 · PART 1 OF 2

End-to-End Pipeline

- **Objective** : Synthesize a robust, memory-efficient data processing pipeline with memory profiling diagnostics.
- Stream rows line-by-line from a CSV data file using lazy generators.
- Enforce validation schemas dynamically by wrapping dictionaries inside Transaction dataclasses.
- Filter out corrupt elements (e.g. invalid negative amounts) and capture validation errors quietly.
- Load valid records into a pandas DataFrame using narrow explicit dtypes and category casting.

ex8_template.py (1/2)

```
"""
Exercise 8: Capstone - Memory-Efficient Data Pipeline
Task: Streaming + validation + compact pandas dtypes + tracemalloc.
"""

import csv, os, tracemalloc
import pandas as pd
from dataclasses import dataclass

@dataclass
class Transaction:
    id_code: str
    category: str
    amount: float

    def __post_init__(self):
        if self.amount < 0:
            raise ValueError("Amount cannot be negative.")

# TODO: Implement step-by-step pipeline segments
def lazy_csv_stream(file_path):
    """Step 1: yield rows line-by-line as raw dicts."""
    pass

def validate_and_filter_stream(records_iterator):
    """Step 2: wrap rows in Transaction, skip invalid ones."""
    pass

def aggregate_and_load_pandas(validated_iterator):
    """Step 3: collect into a DataFrame with compact dtypes."""
    pass
```


End-to-End Pipeline

- **Objective** : Synthesize a robust, memory-efficient data processing pipeline with memory profiling diagnostics.
- Measure peak memory with tracemalloc while the pipeline runs end to end.
- Ensure temporary CSV files are always cleaned up, even if the pipeline raises.

ex8_template.py (2/2)

```
if __name__ == "__main__":
    csv_file = "pipeline_data.csv"
    with open(csv_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["id_code", "category", "amount"])
        for i in range(5000):
            cat = "hardware" if i % 2 == 0 else "software"
            writer.writerow([f"TX_{i}", cat, f"{10.5 * (i % 5)}"])
            # Inject a few invalid negative rows
            if i % 500 == 0:
                bad_row = [f"TX_ERR_{i}", "hardware", "-100.00"]
                writer.writerow(bad_row)

    tracemalloc.start()
    try:
        raw_stream = lazy_csv_stream(csv_file)
        valid_stream = validate_and_filter_stream(raw_stream)
        df_result = aggregate_and_load_pandas(valid_stream)
        current, peak = tracemalloc.get_traced_memory()
        shape = df_result.shape if df_result is not None else None
        print(f"Shape: {shape}")
        print(f"Peak memory: {peak / 1024:.2f} KB")
    finally:
        tracemalloc.stop()
        if os.path.exists(csv_file):
            os.remove(csv_file)
```

EXERCISE 8 · SOLUTION (PART 1 OF 2)

End-to-End Pipeline

- This capstone chains together three ideas from the whole workbook: a lazy generator (Exercise 1/5), dataclass validation (Exercise 3), and compact pandas dtypes (Exercise 7).
- Stage 1, `lazy_csv_stream`, yields raw dictionary rows from disk one at a time — nothing is loaded in bulk.
- Stage 2, `validate_and_filter_stream`, wraps each raw row in a `Transaction` dataclass; any row that fails validation (a negative amount) raises inside `__post_init__` and is quietly skipped.
- Because each stage is a generator, the whole pipeline runs one row through all three stages before moving to the next — not one stage finishing fully before the next begins.

ex8_capstone_pipeline.py (1/2)

```
import csv
import pandas as pd
from dataclasses import dataclass

@dataclass
class Transaction:
    id_code: str
    category: str
    amount: float

    def __post_init__(self):
        if self.amount < 0:
            raise ValueError("Amount cannot be negative.")

def lazy_csv_stream(file_path):
    with open(file_path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            yield row # Stage 1: raw rows, one at a time

def validate_and_filter_stream(records_iterator):
    for row in records_iterator:
        try:
            yield Transaction(row["id_code"], row["category"],
                              float(row["amount"]))
        except ValueError:
            continue # Stage 2: invalid rows skipped quietly
```

EXERCISE 8 · SOLUTION (PART 2 OF 2)

End-to-End Pipeline

- Stage 3, `aggregate_and_load_pandas`, consumes the validated stream and builds the final DataFrame using compact category/float32 dtypes from the start.
- Because the whole chain is generators, memory stays low throughout — `tracemalloc` confirms this by tracking peak allocation across the full run.

ex8_capstone_pipeline.py (2/2)

```
def aggregate_and_load_pandas(validated_iterator):  
    categories, amounts = [], []  
    # Stage 3: consume the validated stream  
    for txn in validated_iterator:  
        categories.append(txn.category)  
        amounts.append(txn.amount)  
    cat_series = pd.Series(categories, dtype="category")  
    amt_series = pd.Series(amounts, dtype="float32")  
    return pd.DataFrame({  
        "category": cat_series,  
        "amount": amt_series,  
    })
```

EXPECTED SHELL OUTPUT

```
DataFrame shape: (5000, 2)  
10 negative-amount rows correctly filtered out of 5010 total
```

SUMMARY

Key Takeaways

- `yield` turns a function into a generator: values are produced lazily, one at a time, instead of all at once.
- `itertools` tools like `islice` and `groupby` operate on iterators directly, avoiding intermediate lists entirely.
- `__post_init__` lets a dataclass validate itself the moment it is created, so invalid objects never exist.
- A custom `json.JSONEncoder.default()` is the standard way to teach `json.dumps()` how to serialise a type it doesn't know.
- Vectorized pandas operations and `groupby.transform()` replace row-by-row loops with fast, columnar expressions.
- Compact dtypes (`int16`, `float32`, `category`) and `memory_usage(deep=True)` turn memory optimisation into something you can measure, not guess at.